Huffman Coding, Time Scheduling & LCS

Algorithms, full C++ code with a line-by-line explanation of every line, and solved variants.

1 Huffman Coding + 3 variants
2 Time Scheduling + 4 variants
3 LCS + 5 variants
4 Quick revision & pitfalls

1 Huffman Coding — Greedy · optimal prefix-free codes

1.1 MAIN ALGORITHM Build Huffman tree, generate codes, encode & decode

Problem. Given characters and their frequencies, find a binary prefix-free code that minimizes the total number of bits Σ f(c)·depth(c).

Idea / answer.

- Put every character in a min-heap keyed by frequency.
- Repeatedly pop the two smallest nodes, join them under a new parent with freq = sum, push the parent back.
- After n−1 merges the heap holds the root. Left edge = 0, right edge = 1; a leaf's path is its codeword.
- Greedy is optimal: the two rarest characters can always be siblings at the deepest level (exchange argument).

Time **O(n log n)** · Space **O(n)** · n = number of distinct characters

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	Pulls in every standard header (iostream, vector, queue, map, string...). GCC-only shortcut, fine for exams/contests.
2	`using namespace std;`	Lets us write `vector`, `cout` instead of `std::vector`, `std::cout`.
3	`struct Node {`	One node of the Huffman tree.
4	`    char ch;`	The character stored here (only meaningful for leaf nodes).
5	`    int freq;`	Frequency of the character, or the sum of frequencies of its subtree for internal nodes.
6	`    Node *left, *right;`	Child pointers. Going left adds bit `0`, going right adds bit `1`.
7	`    Node(char c, int f, Node* l = nullptr, Node* r = nullptr)`	Constructor; children default to `nullptr`, so `Node('a', 5)` creates a leaf.
8	`        : ch(c), freq(f), left(l), right(r) {}`	Member-initializer list copies the arguments into the fields.
9	`};`	End of struct (the `;` is required after a struct definition).
10	`struct Compare {`	Custom comparator object for the priority queue.
11	`    bool operator()(Node* a, Node* b) {`	`priority_queue` calls this to decide which of two nodes has LOWER priority.
12	`        return a->freq > b->freq;`	Using `>` flips the default max-heap into a MIN-heap: smallest frequency stays on top.
13	`    }`	End of operator().
14	`};`	End of Compare.
15	`void buildCodes(Node* root, string code, map<char, string>& codes) {`	DFS that walks the tree and records the bit-string path to every leaf. `codes` is passed by reference so results persist.
16	`    if (!root) return;`	Safety check: nothing to do for an empty subtree.
17	`    if (!root->left && !root->right) {`	No children means this is a leaf, i.e. a real character.
18	`        codes[root->ch] = code.empty() ? "0" : code;`	Store the path. Edge case: if the whole tree is one leaf the path is empty, so give it code "0".
19	`        return;`	Leaves have no children, so stop recursing.

#	C++ code	What this line does
20	`    }`	End of leaf case.
21	`    buildCodes(root->left, code + "0", codes);`	Recurse left, appending `0` to the path.
22	`    buildCodes(root->right, code + "1", codes);`	Recurse right, appending `1` to the path.
23	`}`	End of buildCodes.
24	`Node* buildHuffman(const vector<char>& chars, const vector<int>& freq) {`	Builds the Huffman tree and returns its root.
25	`    priority_queue<Node*, vector<Node*>, Compare> pq;`	Min-heap of node pointers ordered by `freq` (thanks to Compare).
26	`    for (int i = 0; i < (int)chars.size(); i++)`	For every character...
27	`        pq.push(new Node(chars[i], freq[i]));`	...create a leaf and insert it. n pushes = O(n log n).
28	`    while (pq.size() > 1) {`	Keep merging until only one tree remains (exactly n-1 merges).
29	`        Node* x = pq.top(); pq.pop();`	x = node with the smallest frequency; remove it.
30	`        Node* y = pq.top(); pq.pop();`	y = node with the second smallest frequency; remove it.
31	`        pq.push(new Node('$', x->freq + y->freq, x, y));`	New internal node: freq = x+y, x becomes left child, y right child. '$' is just a placeholder char.
32	`    }`	End of merging loop.
33	`    return pq.top();`	The single remaining node is the root of the Huffman tree.
34	`}`	End of buildHuffman.
35	`string decode(Node* root, const string& bits) {`	Converts a bit string back to text by walking the tree.
36	`    if (!root->left && !root->right)`	Edge case: tree is a single leaf (only one distinct character)...
37	`        return string(bits.size(), root->ch);`	...then every bit stands for that character.
38	`    string out;`	Decoded text accumulates here.
39	`    Node* cur = root;`	Start every codeword at the root.
40	`    for (char b : bits) {`	Read the encoded bits one at a time.
41	`        cur = (b == '0') ? cur->left : cur->right;`	`0` moves left, `1` moves right.
42	`        if (!cur->left && !cur->right) {`	Reached a leaf: one full codeword has been read (prefix-free property guarantees no ambiguity).
43	`            out += cur->ch;`	Output that character.
44	`            cur = root;`	Restart from the root for the next codeword.
45	`        }`	End of leaf check.
46	`    }`	End of bit loop.
47	`    return out;`	Return the decoded string.
48	`}`	End of decode.
49	`int main() {`	Program entry point.
50	`    vector<char> chars = {'a', 'b', 'c', 'd', 'e', 'f'};`	The alphabet (classic CLRS example).
51	`    vector<int> freq = {5, 9, 12, 13, 16, 45};`	Frequency of each character (in thousands, per CLRS).
52	`    Node* root = buildHuffman(chars, freq);`	Build the optimal prefix-code tree.
53	`    map<char, string> codes;`	Will hold char → codeword.
54	`    buildCodes(root, "", codes);`	Fill `codes` by DFS starting with an empty path.
55	`    long long totalBits = 0;`	Total encoded length = Σ freq(c) × |code(c)|. `long long` avoids overflow on big inputs.
56	`    for (int i = 0; i < (int)chars.size(); i++) {`	Print each code and accumulate cost.
57	`        cout << chars[i] << " : " << codes[chars[i]] << "\n";`	Show the character and its codeword.
58	`        totalBits += 1LL * freq[i] * codes[chars[i]].size();`	Add freq × code length (1LL forces 64-bit multiplication).
59	`    }`	End of print loop.
60	`    cout << "Total bits = " << totalBits << "\n";`	Print the total cost (the quantity Huffman minimizes).
61	`    string enc = codes['f'] + codes['a'] + codes['c'] + codes['e'];`	Encode the word "face" by concatenating codewords.

#	C++ code	What this line does
62	`    cout << "Encoded face = " << enc << "\n";`	Show the bit string.
63	`    cout << "Decoded = " << decode(root, enc) << "\n";`	Decode it back to verify correctness.
64	`    return 0;`	Signal successful termination.
65	`}`	End of main.

ACTUAL OUTPUT

```
a : 1100
b : 1101
c : 100
d : 101
e : 111
f : 0
Total bits = 224
Encoded face = 01100100111
Decoded = face
```

Hand trace (exam style)

Step	Min-heap (before)	Pop x, y	Push
1	a5 b9 c12 d13 e16 f45	a5, b9	14
2	c12 d13 14 e16 f45	c12, d13	25
3	14 e16 25 f45	14, e16	30
4	25 30 f45	25, 30	55
5	f45 55	f45, 55	100 (root)

Cost = 5·4 + 9·4 + 12·3 + 13·3 + 16·3 + 45·1 = **224** bits (= 14+25+30+55+100, the sum of the pushed values). A fixed 3-bit code would need 100·3 = 300.

Tie-breaking: different tie choices give different codes, but the total cost is always the same optimum. In exams, state your rule (e.g. "smaller on the left").

1.2 VARIANT Optimal merge pattern / minimum cost to connect ropes

Problem. Merge n sorted files of sizes {20, 30, 10, 5, 30} two at a time; merging sizes a and b costs a+b. Find the minimum total cost. (Same problem: connect ropes, combine piles.)

Idea / answer.

- It is **Huffman without the characters**: always merge the two smallest.
- Merges: 5+10=15, 15+20=35, 30+30=60, 35+60=95 → total **205**.

Time **O(n log n)** · Space **O(n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    vector<long long> sizes = {20, 30, 10, 5, 30};`	Sizes of the sorted files (or rope lengths / Huffman frequencies).
5	`    priority_queue<long long, vector<long long>, greater<long long>> pq(`	Min-heap of numbers: `greater<>` puts the smallest on top.
6	`        sizes.begin(), sizes.end());`	Construct the heap directly from the vector in O(n) (heapify).
7	`    long long cost = 0;`	Running total of all merge costs.
8	`    while (pq.size() > 1) {`	Until only one file remains...
9	`        long long a = pq.top(); pq.pop();`	Take the smallest file.
10	`        long long b = pq.top(); pq.pop();`	Take the next smallest file.
11	`        cost += a + b;`	Merging two files of size a and b costs a+b record moves.
12	`        pq.push(a + b);`	The merged file goes back into the pool.
13	`    }`	End of loop (n-1 merges).

#	C++ code	What this line does
14	`    cout << "Minimum merge cost = " << cost << "\n";`	Answer. Same number as Huffman's total encoded bits for these frequencies.
15	`    return 0;`	Done.
16	`}`	End of main.

ACTUAL OUTPUT

```
Minimum merge cost = 205
```

1.3 VARIANT Huffman bits for a text & compression ratio

Problem. For the message "abracadabra", how many bits does Huffman use, compared with ASCII and a fixed-length code? What is the average code length?

Idea / answer.

- Frequencies: a5 b2 r2 c1 d1. Merges 1+1=2, 2+2=4, 2+4=6, 5+6=11 → **23 bits**.
- Fixed length needs $\lceil \log_2 5 \rceil = 3$ bits/char → 33 bits; ASCII 8·11 = 88 bits.
- Average = 23/11 ≈ **2.09 bits/char**.

Time **O(n + k log k)** · k = distinct chars

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    string text = "abracadabra";`	The message we want to compress.
5	`    map<char, int> cnt;`	Frequency table: character → occurrences.
6	`    for (char c : text) cnt[c]++;`	Count every character in one pass.
7	`    priority_queue<int, vector<int>, greater<int>> pq;`	Min-heap of frequencies (we only need costs, not the tree).
8	`    for (auto& [c, f] : cnt) pq.push(f);`	Push each distinct frequency (C++17 structured binding).
9	`    long long huffBits = 0;`	Total Huffman-encoded length.
10	`    if (pq.size() == 1) huffBits = text.size();`	Edge case: one distinct char → each char needs 1 bit.
11	`    while (pq.size() > 1) {`	Standard Huffman merging...
12	`        int a = pq.top(); pq.pop();`	smallest frequency
13	`        int b = pq.top(); pq.pop();`	second smallest
14	`        huffBits += a + b;`	KEY FACT: sum of all merge costs = Σ freq × depth = total encoded bits.
15	`        pq.push(a + b);`	Re-insert merged weight.
16	`    }`	End of merging.
17	`    int k = cnt.size();`	Number of distinct characters.
18	`    int fixedLen = max(1, (int)ceil(log2(k)));`	Fixed-length code needs $\lceil \log_2 k \rceil$ bits per char (at least 1).
19	`    cout << "Distinct chars = " << k << "\n";`	Report k.
20	`    cout << "ASCII bits     = " << 8 * text.size() << "\n";`	Uncompressed: 8 bits per character.
21	`    cout << "Fixed bits     = " << fixedLen * text.size() << "\n";`	Fixed-length coding cost.
22	`    cout << "Huffman bits   = " << huffBits << "\n";`	Optimal variable-length cost.
23	`    cout << fixed << setprecision(2);`	Print floating values with 2 decimals.
24	`    cout << "Avg bits/char  = " << (double)huffBits / text.size() << "\n";`	Average code length = total bits / message length.
25	`    cout << "Compression vs ASCII = "`	Ratio label...
26	`         << 100.0 * huffBits / (8 * text.size()) << "%\n";`	...percentage of the original size that remains.
27	`    return 0;`	Done.
28	`}`	End of main.

```
ACTUAL OUTPUT
Distinct chars = 5
ASCII bits     = 88
Fixed bits     = 33
Huffman bits   = 23
Avg bits/char  = 2.09
Compression vs ASCII = 26.14%
```

1.4 VARIANT Check whether a code is prefix-free

Problem. Given a set of binary codewords, decide if any codeword is a prefix of another (which would make decoding ambiguous).

Idea / answer.

- Sort the codewords; a prefix always sorts immediately before a word that extends it, so checking adjacent pairs is enough.
- {0, 101, 100, 111, 1101, 1100} → **prefix-free**; {0, 01, 11} → **not** (0 is a prefix of 01).

Time **O(n·L log n)** · L = max code length

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`bool isPrefixFree(vector<string> codes) {`	Returns true if no codeword is a prefix of another (copy is taken because we sort it).
4	`    sort(codes.begin(), codes.end());`	Lexicographic sort: if A is a prefix of B, A ends up right before something starting with A.
5	`    for (int i = 0; i + 1 < (int)codes.size(); i++) {`	Compare each adjacent pair only — enough after sorting.
6	`        const string& a = codes[i];`	Current code.
7	`        const string& b = codes[i + 1];`	Next code in sorted order.
8	`        if (b.compare(0, a.size(), a) == 0) return false;`	First |a| chars of b equal a → a is a prefix of b → not prefix-free.
9	`    }`	End of loop.
10	`    return true;`	No violations found.
11	`}`	End of function.
12	`int main() {`	Entry point.
13	`    vector<string> c1 = {"0", "101", "100", "111", "1101", "1100"};`	The Huffman codes from Program 1.
14	`    vector<string> c2 = {"0", "01", "11"};`	"0" is a prefix of "01" → ambiguous.
15	`    cout << boolalpha;`	Print bools as true/false instead of 1/0.
16	`    cout << "c1 prefix-free? " << isPrefixFree(c1) << "\n";`	Expected: true.
17	`    cout << "c2 prefix-free? " << isPrefixFree(c2) << "\n";`	Expected: false.
18	`    return 0;`	Done.
19	`}`	End of main.

```
ACTUAL OUTPUT
c1 prefix-free? true
c2 prefix-free? false
```

2 Time Scheduling Activity selection & interval problems

2.1 MAIN ALGORITHM Activity selection (max non-overlapping intervals)

Problem. n activities with start s_i and finish f_i share one resource. Select the maximum number of mutually compatible activities.

Idea / answer.

- Sort by **finish time**.
- Pick the first; then pick each next activity whose start ≥ finish of the last picked.
- Answer for the CLRS data: **{a1, a4, a8, a11}**, size 4.

Time **O(n log n)** (O(n) if already sorted) · Space **O(n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    vector<int> s = {1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12};`	Start times of activities a1..a11 (CLRS example).
5	`    vector<int> f = {4, 5, 6, 7, 9, 9, 10, 11, 12, 14, 16};`	Finish times of the same activities.
6	`    int n = s.size();`	Number of activities.
7	`    vector<int> idx(n);`	We sort indices (not the arrays) so we still know the original activity number.
8	`    iota(idx.begin(), idx.end(), 0);`	Fill idx with 0,1,2,...,n-1.
9	`    sort(idx.begin(), idx.end(), [&](int a, int b) {`	Sort indices with a lambda comparator...
10	`        return f[a] < f[b];`	...by increasing FINISH time — the greedy choice that is provably optimal.
11	`    });`	End of sort call.
12	`    vector<int> chosen;`	Selected activities (1-based numbers).
13	`    int lastFinish = INT_MIN;`	Finish time of the last selected activity; INT_MIN so the first one always fits.
14	`    for (int i : idx) {`	Scan activities in finish-time order.
15	`        if (s[i] >= lastFinish) {`	Compatible if it starts at/after the last chosen one ends (use `>` if touching is NOT allowed).
16	`            chosen.push_back(i + 1);`	Take it (store 1-based label).
17	`            lastFinish = f[i];`	Update the "busy until" time.
18	`        }`	End of compatibility check.
19	`    }`	End of scan. Total O(n log n) due to sorting.
20	`    cout << "Max activities = " << chosen.size() << "\n";`	Size of the optimal set.
21	`    cout << "Chosen: ";`	Label.
22	`    for (int a : chosen) cout << "a" << a << " ";`	Print each chosen activity.
23	`    cout << "\n";`	Newline.
24	`    return 0;`	Done.
25	`}`	End of main.

ACTUAL OUTPUT

```
Max activities = 4
Chosen: a1 a4 a8 a11
```

Hand trace (sorted by finish time)

Act.	a1	a2	a3	a4	a5	a6	a7	a8	a9	a10	a11
s, f	1,4	3,5	0,6	5,7	3,9	5,9	6,10	8,11	8,12	2,14	12,16
take?	✓	×	×	✓	×	×	×	✓	×	×	✓

Why finish-time? Exchange argument: an optimal solution O can swap its first activity for the earliest-finishing one without conflicts, so greedy's first pick is always safe; repeat on the rest. Sorting by start time or by shortest duration both have counter-examples.

2.2 VARIANT Weighted interval scheduling (max profit)

Problem. Each job has (start, end, profit). Choose non-overlapping jobs with maximum total profit. Greedy fails here — use DP.

Idea / answer.

- Sort by end time. Let p(i) = number of earlier jobs that end ≤ start of job i (binary search).
- dp[i] = max(dp[i−1], profit_i + dp[p(i)]).
- Jobs (1,3,50) (2,4,10) (3,5,40) (3,6,70): take (1,3,50)+(3,6,70) = **120**.

Time **O(n log n)** · Space **O(n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`struct Job { int s, e, p; };`	A job: start time, end time, profit (weight).
4	`int main() {`	Entry point.
5	`    vector<Job> jobs = {{1, 3, 50}, {2, 4, 10}, {3, 5, 40}, {3, 6, 70}};`	Sample input (LeetCode 1235 example 1).
6	`    int n = jobs.size();`	Number of jobs.
7	`    sort(jobs.begin(), jobs.end(),`	Sort jobs...
8	`          [](const Job& a, const Job& b) { return a.e < b.e; });`	...by end time, so "previous compatible job" is a prefix.
9	`    vector<int> ends(n);`	Separate array of end times for binary search.
10	`    for (int i = 0; i < n; i++) ends[i] = jobs[i].e;`	Copy end times (already sorted).
11	`    vector<long long> dp(n + 1, 0);`	dp[i] = max profit using only the first i jobs; dp[0] = 0.
12	`    for (int i = 1; i <= n; i++) {`	Fill dp left to right.
13	`        const Job& j = jobs[i - 1];`	The i-th job (0-based index i-1).
14	`        int k = upper_bound(ends.begin(), ends.begin() + (i - 1), j.s)`	Binary search among earlier jobs for the first end > start...
15	`                - ends.begin();`	...so k = count of earlier jobs with end ≤ start (all compatible). This is p(i).
16	`        dp[i] = max(dp[i - 1], dp[k] + j.p);`	Either skip job i, or take it plus the best of its compatible prefix.
17	`    }`	End of DP. O(n log n) overall.
18	`    cout << "Max profit = " << dp[n] << "\n";`	Answer.
19	`    return 0;`	Done.
20	`}`	End of main.

ACTUAL OUTPUT
```
Max profit = 120
```

2.3 VARIANT Job sequencing with deadlines (unit-time jobs)

Problem. Each job takes 1 unit of time and earns its profit only if finished by its deadline. Maximize profit. Jobs: a(2,100) b(1,19) c(2,27) d(1,25) e(3,15).

Idea / answer.

- Sort by profit (descending). Put each job in the **latest free slot ≤ its deadline**; skip if none.
- a→slot 2, c→slot 1, d,b rejected (slot 1 taken), e→slot 3. Sequence **c, a, e**, profit **142**.

Time **O(n log n + n·D)** (D = max deadline); O(n log n) with DSU

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`struct Job { char id; int deadline, profit; };`	Each job takes 1 unit of time, must finish by `deadline`, earns `profit`.
4	`int main() {`	Entry point.
5	`    vector<Job> jobs = {{'a', 2, 100}, {'b', 1, 19}, {'c', 2, 27},`	Sample jobs...
6	`                        {'d', 1, 25}, {'e', 3, 15}};`	...(continued).
7	`    sort(jobs.begin(), jobs.end(), [](const Job& x, const Job& y) {`	Greedy order:
8	`        return x.profit > y.profit;`	most profitable job first.
9	`    });`	End of sort.
10	`    int maxD = 0;`	Largest deadline = number of time slots we need.
11	`    for (auto& j : jobs) maxD = max(maxD, j.deadline);`	Find it.
12	`    vector<char> slot(maxD + 1, '-');`	slot[t] = job done in time slot t (1-based); '-' means free.
13	`    int total = 0;`	Accumulated profit.
14	`    for (auto& j : jobs) {`	Consider jobs by decreasing profit.
15	`        for (int t = j.deadline; t >= 1; t--) {`	Try the LATEST free slot not after the deadline (keeps earlier slots open for others).
16	`            if (slot[t] == '-') {`	Free slot found...
17	`                slot[t] = j.id;`	...schedule the job there,
18	`                total += j.profit;`	collect its profit,
19	`                break;`	and stop searching for this job.
20	`            }`	End if.
21	`        }`	If no slot was free, the job is simply rejected.
22	`    }`	End of greedy loop. O(n log n + n·maxD); DSU makes it ~O(n log n).
23	`    cout << "Sequence: ";`	Label.
24	`    for (int t = 1; t <= maxD; t++) cout << slot[t] << " ";`	Print jobs in time order.
25	`    cout << "\nMax profit = " << total << "\n";`	Answer.
26	`    return 0;`	Done.
27	`}`	End of main.

ACTUAL OUTPUT

```
Sequence: c a e
Max profit = 142
```

2.4 VARIANT Interval partitioning (minimum rooms / platforms)

Problem. Schedule ALL intervals using as few rooms as possible (classic: minimum railway platforms). Arrivals 900 940 950 1100 1500 1800, departures 910 1200 1120 1130 1900 2000.

Idea / answer.

- Minimum rooms = **maximum number of intervals overlapping at any time** (the depth).
- Sort starts and ends separately and sweep: start → +1, end → −1; track the peak. Answer **3**.
- Alternative: sort by start and keep a min-heap of room end times.

Time **O(n log n)** · Space **O(1)** extra

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    vector<int> arr = {900, 940, 950, 1100, 1500, 1800};`	Start (arrival) times — e.g. trains at a station.
5	`    vector<int> dep = {910, 1200, 1120, 1130, 1900, 2000};`	End (departure) times, paired by index with arr.
6	`    sort(arr.begin(), arr.end());`	Sort starts independently...
7	`    sort(dep.begin(), dep.end());`	...and ends independently (we only count events, pairing doesn't matter).
8	`    int n = arr.size(), i = 0, j = 0;`	i walks starts, j walks ends.
9	`    int cur = 0, best = 0;`	cur = rooms in use now, best = maximum seen.
10	`    while (i < n) {`	Sweep through time while starts remain.
11	`        if (arr[i] <= dep[j]) {`	Next event is a start (≤ means an interval ending at the same time still occupies the room).
12	`            cur++; i++;`	One more room needed; advance start pointer.
13	`        } else {`	Next event is an end...
14	`            cur--; j++;`	...a room becomes free; advance end pointer.
15	`        }`	End if/else.
16	`        best = max(best, cur);`	Track the peak overlap = minimum rooms required (= depth).
17	`    }`	End sweep. O(n log n).
18	`    cout << "Minimum rooms/platforms = " << best << "\n";`	Answer.
19	`    return 0;`	Done.
20	`}`	End of main.

ACTUAL OUTPUT
```
Minimum rooms/platforms = 3
```

2.5 VARIANT Scheduling to minimize maximum lateness

Problem. Jobs have processing time t_j and deadline d_j, run on one machine. Lateness = max(0, finish − deadline). Minimize the maximum lateness.

Idea / answer.

- **Earliest Deadline First**: sort by deadline, run back-to-back with no idle time.
- Proof idea: any schedule with an inversion (later deadline first) can swap adjacent jobs without increasing max lateness.
- Kleinberg–Tardos example → max lateness **1**.

Time **O(n log n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    vector<pair<int, int>> jobs = {{3, 6}, {2, 8}, {1, 9},`	(processing time t, deadline d) — Kleinberg–Tardos example...
5	`                                  {4, 9}, {3, 14}, {2, 15}};`	...(continued).
6	`    sort(jobs.begin(), jobs.end(), [](auto& a, auto& b) {`	Earliest-Deadline-First (EDF) order:
7	`        return a.second < b.second;`	sort by deadline ascending.
8	`    });`	End of sort.
9	`    int time = 0, maxLate = 0;`	`time` = current clock; `maxLate` = worst lateness so far.
10	`    for (auto& [t, d] : jobs) {`	Run jobs back-to-back with no idle time.
11	`        int start = time;`	This job starts now.

#	C++ code	What this line does
12	`        time += t;`	It finishes after t units.
13	`        int late = max(0, time - d);`	Lateness = how far past the deadline it finished (never negative).
14	`        maxLate = max(maxLate, late);`	Keep the maximum.
15	`        cout << "[" << start << "," << time << ") d=" << d`	Print the job's interval and deadline...
16	`             << " late=" << late << "\n";`	...and its lateness.
17	`    }`	End loop. O(n log n).
18	`    cout << "Minimum possible max lateness = " << maxLate << "\n";`	EDF is optimal for minimizing maximum lateness.
19	`    return 0;`	Done.
20	`}`	End of main.

ACTUAL OUTPUT

```
[0,3) d=6 late=0
[3,5) d=8 late=0
[5,6) d=9 late=0
[6,10) d=9 late=1
[10,13) d=14 late=0
[13,15) d=15 late=0
Minimum possible max lateness = 1
```

3 Longest Common Subsequence Dynamic programming

3.1 MAIN ALGORITHM LCS length + reconstruct the sequence

Problem. Given X and Y, find the longest sequence that appears in both in the same order (not necessarily contiguous).

Idea / answer.

- Optimal substructure: if last chars match, they belong to the LCS; otherwise drop one last char and take the better result.
- Fill an (m+1)×(n+1) table row by row; backtrack from c[m][n] to print one LCS.
- X = ABCBDAB, Y = BDCABA → length **4**, e.g. **BCBA**.

Time **O(m·n)** · Space **O(m·n)** · Backtrack **O(m+n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    string X = "ABCBDAB", Y = "BDCABA";`	Two input sequences (CLRS example).
5	`    int m = X.size(), n = Y.size();`	Their lengths.
6	`    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));`	(m+1)×(n+1) table; dp[i][j] = LCS length of X[0..i-1] and Y[0..j-1]. Row 0 / col 0 = empty prefix = 0.
7	`    for (int i = 1; i <= m; i++) {`	For each prefix length of X...
8	`        for (int j = 1; j <= n; j++) {`	...and each prefix length of Y.
9	`            if (X[i - 1] == Y[j - 1])`	Last characters match (strings are 0-based, table is 1-based)...
10	`                dp[i][j] = dp[i - 1][j - 1] + 1;`	...extend the LCS of both shorter prefixes by 1 (diagonal ↖).
11	`            else`	Last characters differ...
12	`                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);`	...drop the last char of X (↑) or of Y (←), keep the better.
13	`        }`	End inner loop.
14	`    }`	End of table fill. O(m·n) time and space.
15	`    string lcs;`	Will hold one actual LCS (built backwards).
16	`    int i = m, j = n;`	Start backtracking from the bottom-right cell.
17	`    while (i > 0 && j > 0) {`	Stop when either prefix becomes empty.
18	`        if (X[i - 1] == Y[j - 1]) {`	A match here means this char is part of the LCS.
19	`            lcs += X[i - 1];`	Record it.
20	`            i--; j--;`	Move diagonally.
21	`        } else if (dp[i - 1][j] >= dp[i][j - 1]) {`	Otherwise follow the larger neighbour; ties go UP (CLRS convention)...
22	`            i--;`	...move up.
23	`        } else {`	Left neighbour is strictly larger...
24	`            j--;`	...move left.
25	`        }`	End of choice.
26	`    }`	End backtrack. O(m+n).
27	`    reverse(lcs.begin(), lcs.end());`	Characters were collected last-to-first, so reverse.
28	`    cout << "LCS length = " << dp[m][n] << "\n";`	The answer is in the bottom-right cell.
29	`    cout << "One LCS    = " << lcs << "\n";`	Print the reconstructed subsequence.
30	`    return 0;`	Done.
31	`}`	End of main.

ACTUAL OUTPUT

```
LCS length = 4
One LCS    = BCBA
```

DP table (X = ABCBDAB down, Y = BDCABA across). Shaded = backtrack path; dark = matched characters → B C B A.

	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	↑0	↑0	↑0	↖1	←1	↖1
B	0	↖**1**	←1	←1	↑1	↖2	←2
C	0	↑1	↑1	↖**2**	←2	↑2	↑2
B	0	↖1	↑1	↑2	↑2	↖**3**	←3
D	0	↑1	↖2	↑2	↑2	↑3	↑3
A	0	↑1	↑2	↑2	↖3	↑3	↖**4**
B	0	↖1	↑2	↑2	↑3	↖4	↑4

Recurrence: c[i][j] = 0 if i=0 or j=0; c[i−1][j−1]+1 if x_i=y_j; max(c[i−1][j], c[i][j−1]) otherwise. Several LCSs can exist (e.g. BDAB, BCAB) — they all have length 4.

3.2 VARIANT Longest common SUBSTRING (contiguous)

Problem. Find the longest string that appears contiguously in both X = ABABC and Y = BABCA.

Idea / answer.

- Same table, but on mismatch set dp = 0 (the run is broken); answer = max cell, not the corner.
- Answer **BABC**, length 4.

Time **O(m·n)** · Space **O(m·n)** (O(n) with two rows)

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    string X = "ABABC", Y = "BABCA";`	Inputs.
5	`    int m = X.size(), n = Y.size();`	Lengths.
6	`    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));`	dp[i][j] = length of longest common SUFFIX of X[0..i-1] and Y[0..j-1].
7	`    int best = 0, endX = 0;`	best length so far, and the index in X where that substring ends.
8	`    for (int i = 1; i <= m; i++) {`	Every prefix of X.
9	`        for (int j = 1; j <= n; j++) {`	Every prefix of Y.
10	`            if (X[i - 1] == Y[j - 1]) {`	Characters match → the common suffix grows.
11	`                dp[i][j] = dp[i - 1][j - 1] + 1;`	Extend diagonally.
12	`                if (dp[i][j] > best) {`	New record?
13	`                    best = dp[i][j];`	Save its length...
14	`                    endX = i;`	...and where it ends in X.
15	`                }`	End record check.
16	`            }`	On mismatch dp stays 0 — THE difference from LCS (contiguity breaks).
17	`        }`	End inner loop.
18	`    }`	End fill. O(m·n).
19	`    cout << "Longest common substring = "`	Label...
20	`         << X.substr(endX - best, best)`	...extract it: starts at endX-best, length best.
21	`         << " (length " << best << ")\n";`	Print length.
22	`    return 0;`	Done.
23	`}`	End of main.

```
ACTUAL OUTPUT
Longest common substring = BABC (length 4)
```

3.3 VARIANT Space-optimized LCS length (two rows)

Problem. Compute only the LCS length when memory is limited (e.g. strings of length 10^5 where an m×n table won't fit).

Idea / answer.

- Row i depends only on row i−1, so keep two 1-D arrays and swap them.
- Use the shorter string as the row → O(min(m,n)) memory. Reconstruction needs Hirschberg's algorithm.

Time **O(m·n)** · Space **O(min(m,n))**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int lcsLength(const string& X, const string& Y) {`	Returns only the LCS length using O(min(m,n)) memory.
4	`    if (X.size() < Y.size()) return lcsLength(Y, X);`	Make Y the shorter string so the rows are as small as possible.
5	`    int n = Y.size();`	Row width.
6	`    vector<int> prev(n + 1, 0), cur(n + 1, 0);`	prev = row i-1, cur = row i. Each cell only needs these two rows.
7	`    for (char c : X) {`	One iteration per character of X (= one table row).
8	`        for (int j = 1; j <= n; j++) {`	Fill the current row.
9	`            if (c == Y[j - 1]) cur[j] = prev[j - 1] + 1;`	Match: diagonal + 1.
10	`            else cur[j] = max(prev[j], cur[j - 1]);`	Mismatch: max of up (prev[j]) and left (cur[j-1]).
11	`        }`	End row.
12	`        swap(prev, cur);`	Current row becomes the previous row for the next iteration (O(1) swap).
13	`    }`	End of rows.
14	`    return prev[n];`	After the final swap the last computed row is in `prev`.
15	`}`	End function. Time O(m·n), space O(min(m,n)). Cannot reconstruct the string directly.
16	`int main() {`	Entry point.
17	`    cout << lcsLength("AGGTAB", "GXTXAYB") << "\n";`	Expected 4 ("GTAB").
18	`    cout << lcsLength("ABCBDAB", "BDCABA") << "\n";`	Expected 4.
19	`    return 0;`	Done.
20	`}`	End of main.

```
ACTUAL OUTPUT
4
4
```

3.4 VARIANT Shortest common supersequence & insert/delete conversion

Problem. (a) Length of the shortest string having both AGGTAB and GXTXAYB as subsequences. (b) Minimum insertions and deletions to turn "heap" into "pea".

Idea / answer.

- (a) $|SCS| = m + n - LCS = 6 + 7 - 4 = \mathbf{9}$.
- (b) LCS(heap, pea) = "ea" = 2 → deletions = 4 − 2 = **2** (h, p), insertions = 3 − 2 = **1** (p).

Time **O(m·n)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int lcs(const string& a, const string& b) {`	Standard LCS length (full table version).

#	C++ code	What this line does
4	`    vector<vector<int>> dp(a.size() + 1, vector<int>(b.size() + 1, 0));`	DP table with zero borders.
5	`    for (size_t i = 1; i <= a.size(); i++)`	Rows.
6	`        for (size_t j = 1; j <= b.size(); j++)`	Columns.
7	`            dp[i][j] = (a[i - 1] == b[j - 1]) ? dp[i - 1][j - 1] + 1`	Match → diagonal + 1,
8	`                        : max(dp[i - 1][j], dp[i][j - 1]);`	else best of up/left.
9	`    return dp[a.size()][b.size()];`	Bottom-right cell.
10	`}`	End lcs.
11	`int main() {`	Entry point.
12	`    string A = "AGGTAB", B = "GXTXAYB";`	Inputs for Shortest Common Supersequence.
13	`    int L = lcs(A, B);`	Shared part — needs to be written only once.
14	`    cout << "SCS length = " << A.size() + B.size() - L << "\n";`	|SCS| = m + n − LCS (common chars counted once).
15	`    string from = "heap", to = "pea";`	Convert `from` into `to` using only insertions & deletions.
16	`    int k = lcs(from, to);`	Characters we can keep.
17	`    cout << "Deletions  = " << from.size() - k << "\n";`	Delete everything in `from` not in the LCS.
18	`    cout << "Insertions = " << to.size() - k << "\n";`	Insert everything in `to` not in the LCS.
19	`    return 0;`	Done.
20	`}`	End of main.

```
ACTUAL OUTPUT
SCS length = 9
Deletions  = 2
Insertions = 1
```

3.5 VARIANT Longest palindromic subsequence & min insertions

Problem. Find the longest palindromic subsequence of "BBABCBCAB" and the fewest insertions needed to make the string a palindrome.

Idea / answer.

- LPS(s) = **LCS(s, reverse(s))** → 7 ("BABCBAB").
- Min insertions = n − LPS = 9 − 7 = **2**.

Time **O(n²)** · Space **O(n²)**

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers.
2	`using namespace std;`	Avoid `std::` prefixes.
3	`int main() {`	Entry point.
4	`    string s = "BBABCBCAB";`	Input string.
5	`    string r(s.rbegin(), s.rend());`	r = reverse of s (built from reverse iterators).
6	`    int n = s.size();`	Length.
7	`    vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));`	LCS table for s vs r.
8	`    for (int i = 1; i <= n; i++)`	Rows (prefixes of s).
9	`        for (int j = 1; j <= n; j++)`	Columns (prefixes of reversed s).
10	`            dp[i][j] = (s[i - 1] == r[j - 1]) ? dp[i - 1][j - 1] + 1`	Match → diagonal + 1,
11	`                        : max(dp[i - 1][j], dp[i][j - 1]);`	else max(up, left).
12	`    int lps = dp[n][n];`	LPS(s) = LCS(s, reverse(s)).
13	`    cout << "Longest palindromic subsequence = " << lps << "\n";`	Print LPS length.
14	`    cout << "Min insertions to make palindrome = " << n - lps << "\n";`	Every char outside the LPS needs a mirror partner inserted.

#	C++ code	What this line does
15	`    return 0;`	Done.
16	`}`	End of main.

ACTUAL OUTPUT

```
Longest palindromic subsequence = 7
Min insertions to make palindrome = 2
```

3.6 VARIANT LCS of words in two sentences

Problem. Find the longest sequence of WORDS that appears in both sentences in the same order. S1 = "the quick brown fox jumps over the lazy dog", S2 = "the brown dog jumps over a lazy fox".

Idea / answer.

- Split each sentence into a `vector<string>` of words, then run the normal LCS DP with words as the symbols (compare with `==` on strings).
- Answer: **5** words → **the brown jumps over lazy**.
- Variations: lower-case both sentences and strip punctuation first for a case-insensitive match. Use the substring version (3.2) on words to find the longest common contiguous phrase.

Time **O(m·n·L)** · m, n = word counts, L = average word length (string compare)

#	C++ code	What this line does
1	`#include <bits/stdc++.h>`	All standard headers (includes <sstream> for stringstream).
2	`using namespace std;`	Avoid `std::` prefixes.
3	`vector<string> splitWords(const string& s) {`	Breaks a sentence into a list of words (tokens).
4	`    vector<string> words;`	Result list.
5	`    stringstream ss(s);`	Treat the sentence like an input stream.
6	`    string w;`	Holds one word at a time.
7	`    while (ss >> w) words.push_back(w);`	`>>` reads up to the next whitespace, so multiple spaces/tabs are skipped automatically.
8	`    return words;`	Return the token list.
9	`}`	End of splitWords.
10	`int main() {`	Entry point.
11	`    string s1 = "the quick brown fox jumps over the lazy dog";`	First sentence.
12	`    string s2 = "the brown dog jumps over a lazy fox";`	Second sentence.
13	`    vector<string> A = splitWords(s1), B = splitWords(s2);`	Convert both sentences to word arrays — now each "character" of LCS is a whole word.
14	`    int m = A.size(), n = B.size();`	Number of words in each sentence.
15	`    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));`	dp[i][j] = LCS length (in words) of the first i words of A and first j words of B.
16	`    for (int i = 1; i <= m; i++) {`	Each word-prefix of A.
17	`        for (int j = 1; j <= n; j++) {`	Each word-prefix of B.
18	`            if (A[i - 1] == B[j - 1])`	Whole-word comparison: string `==` checks the words are identical.
19	`                dp[i][j] = dp[i - 1][j - 1] + 1;`	Same word → extend diagonally.
20	`            else`	Different words...
21	`                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);`	...skip a word from A or from B, keep the better.
22	`        }`	End inner loop.
23	`    }`	End fill. O(m·n) word comparisons.
24	`    vector<string> res;`	Words of one LCS, collected backwards.
25	`    int i = m, j = n;`	Backtrack from the bottom-right cell.
26	`    while (i > 0 && j > 0) {`	Until a sentence is exhausted.
27	`        if (A[i - 1] == B[j - 1]) {`	Matching word belongs to the answer.

#	C++ code	What this line does
28	`            res.push_back(A[i - 1]);`	Record it.
29	`            i--; j--;`	Move diagonally.
30	`        } else if (dp[i - 1][j] >= dp[i][j - 1]) {`	Otherwise follow the larger neighbour (ties go up)...
31	`            i--;`	...move up.
32	`        } else {`	Left is strictly larger...
33	`            j--;`	...move left.
34	`        }`	End of choice.
35	`    }`	End backtrack.
36	`    reverse(res.begin(), res.end());`	Restore the original word order.
37	`    cout << "LCS length (words) = " << dp[m][n] << "\n";`	Answer as a word count.
38	`    cout << "Common words: ";`	Label.
39	`    for (auto& w : res) cout << w << " ";`	Print the common word subsequence.
40	`    cout << "\n";`	Newline.
41	`    return 0;`	Done.
42	`}`	End of main.

ACTUAL OUTPUT

```
LCS length (words) = 5
Common words: the brown jumps over lazy
```

4 Quick revision

Problem	Technique	Key rule	Time
Huffman coding	Greedy + min-heap	Merge two smallest, n−1 times	O(n log n)
Optimal merge / ropes	Greedy + min-heap	Cost = Σ merged sums	O(n log n)
Activity selection	Greedy	Sort by finish, take if s ≥ last f	O(n log n)
Weighted intervals	DP + binary search	dp[i]=max(dp[i−1], w_i+dp[p(i)])	O(n log n)
Job sequencing	Greedy	Max profit first, latest free slot	O(n log n + nD)
Min rooms	Sweep line	Peak overlap	O(n log n)
Min max lateness	Greedy (EDF)	Sort by deadline	O(n log n)
LCS	DP	match → ↖+1, else max(↑,←)	O(mn)
Common substring	DP	mismatch → 0, answer = max cell	O(mn)
SCS / ins-del	LCS reduction	m+n−L ; (m−L) del, (n−L) ins	O(mn)
Word-level LCS	DP on tokens	split into words, same recurrence	O(mn·L)
Palindromic subseq.	LCS reduction	LCS(s, rev s); insert n−LPS	O(n^2)

Common exam pitfalls

- `priority_queue` is a **max**-heap by default — use `greater<>` or a custom comparator with `>` for Huffman.
- Huffman with one distinct character: give it code "0", otherwise the code is empty.
- Activity selection: decide whether touching intervals (s = f) are compatible — `>=` vs `>`.
- Greedy by profit/weight does **not** work for weighted interval scheduling — that one needs DP.
- LCS table is 1-based, strings are 0-based: compare `X[i-1]` with `Y[j-1]`.
- Subsequence ≠ substring: substring resets to 0 on mismatch.
- When backtracking LCS, the tie rule (↑ vs ←) changes *which* LCS you print, never its length.

All programs were compiled with `g++ -std=c++17 -Wall` and the outputs shown are from actual runs.